9-1 用户注册

需求场景

对于很多应用来说，注册接口是必可不少的。想要实现注册接口，其实很简单，但是你会发现会有很多种选择：

- 注册时有需要哪些字段？
- 注册接口使用什么请求方式？
- 前端朝后端传数据时，放在查询参数中、请求体中？
- 如果放在请求体中，使用JSON格式还是表单格式？

解决方式

你会发现就一个简单的注册接口，其实还是有很多问题需要我们思考的。

首先，注册接口一般是将用户的个人信息提交给服务端，因此，我们选择POST请求

然后，注册信息中一般都包含密码，所以不能简单的在查询参数中提交给后端，需要把数据放在请求体中。

最后，请求体如何使用JSON格式，那注册时如果有上传文件的需求，将比较麻烦。因此我们使用Form表单的形式。

结论：对于常见的注册接口，使用POST，使用Form表单来上传数据。

补充：任何一种方式都可以实现，但最终如何选择还是要看业务需求。

前端页面

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Title</title>
</head>
<body>

<h1>注册</h1>
<form action="http://127.0.0.1:8000/register" method="post"
enctype="multipart/form-data">
    <p>用户名: <input type="text" name="username"></p>
    <p>密码: <input type="password" name="password"></p>
    <p>确认密码: <input type="password" name="re_password"></p>
    <p>邮件: <input type="email" name="email"></p>
    <p><input type="submit"></p>

</form>

</body>
</html>
```

后端接口

- 注册逻辑：判断用户名是否已经注册；判断二次密码是否一致。
- 缺陷：密码没有加密；数据没有持久化（存数据库）。

```
from fastapi import FastAPI, Form, Depends, HTTPException

app = FastAPI(title="登录认证相关")


# 模拟数据库
USERS = {}


def get_form_data(username: str = Form(), password: str = Form(), re_password:
str = Form(), email: str = Form()):
    return {
        "username": username,
        "password": password,
        "re_password": re_password,
        "email": email,
    }


@app.post("/register")
def register(form_data: dict = Depends(get_form_data)):
    # 判断用户名是否已存在
    if form_data["username"] in USERS:
        raise HTTPException(detail="用户名已经存在", status_code=400)
    # 判断密码是否一致
    if form_data["password"] != form_data["re_password"]:
        raise HTTPException(detail="两次密码输入不一致", status_code=400)
    # 保存用户信息，完成注册
    USERS[form_data["username"]] = form_data

    # 返回新用户基本信息
    return {"username": form_data["username"], "email": form_data["email"]}
```

9-2 用户密码加密

存用户信息时，不能明文存储，一定要做加密处理。

示例1： 使用python内置库hasslib

```
import hashlib


m = hashlib.md5("盐".encode("utf-8"))     # 支持加盐
m.update("hello".encode("utf-8"))

print(m.digest())        # 加密后的二进制文本
print(m.hexdigest())     # 以16进制形式返回加密内容
```

示例2：使用第三方库 passlib

> pip install "passlib[bcrypt]"

```
from passlib.context import CryptContext

crypt = CryptContext(schemes=["bcrypt"], deprecated="auto")

crypt.hash("hello")      # 加密
```

9-3 集成MySQL的注册

需求场景

上节课，我们实现了注册页面和注册接口，但是有明显的缺陷，比如说密码没有加密，数据没有存数据库。

解决方式

第一步：数据持久化

- 新建用户表

```
CREATE TABLE `users` (
    `id` int(11) NOT NULL AUTO_INCREMENT,
    `username` varchar(255) NOT NULL,
    `password` varchar(255) NOT NULL,
    `email` varchar(255) NOT NULL,
    PRIMARY KEY (`id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_bin AUTO_INCREMENT=1 ;
```

- 使用SQLAlchemy的ORM操作数据库（基于8-13节代码结构，其他模块代码见源文件）

```
from fastapi import FastAPI, Form, Depends, HTTPException
```

```python
from sqlalchemy.orm import Session
from passlib.context import CryptContext

from database import get_db
from models import User
from schemas import UserOut

app = FastAPI(title="登录认证相关")


crypt = CryptContext(schemes=["bcrypt"], deprecated="auto")


class UserForm:
    def __init__(self, username: str = Form(), password: str = Form(),
re_password: str = Form(), email: str = Form()):
        self.username = username
        self.password = password
        self.re_password = re_password
        self.email = email


@app.post("/register", response_model=UserOut)
def register(user: UserForm = Depends(), db: Session = Depends(get_db)):
    # 注册接口核心部分，此部分没有放在crud.py文件中， 大家可以自行处理下
    # 判断用户名是否存在
    if db.query(User).filter(User.username == user.username).first():
        raise HTTPException(detail="用户名已存在", status_code=400)
    if user.password != user.re_password:
        raise HTTPException(detail="两次密码输入不一致", status_code=400)

    # 新增用户
    new_user = User(
        username=user.username,
        password=crypt.hash(user.password),     # 密文存储
        email=user.email
    )
    db.add(new_user)
    db.commit()
    db.refresh(new_user)
    return new_user
```

9-4 用户登录

登录接口的作用：通过用户输入用户名和密码，找到该用户在本网站上的基本信息。

登录接口的核心逻辑：校验用户名和密码是否和数据库中保存的像匹配，匹配则登录成功，否则失败。

代码示例

```python
from fastapi import FastAPI, Form, Depends, HTTPException
```

```python
from sqlalchemy.orm import Session
from passlib.context import CryptContext

from database import get_db
from models import User
from schemas import UserOut

app = FastAPI(title="登录认证相关")


crypt = CryptContext(schemes=["bcrypt"], deprecated="auto")


@app.post("/login", response_model=UserOut)
def login(username: str = Form(), password: str = Form(), db: Session =
Depends(get_db)):
    # 根据用户名找该用户是否存在
    db_user: User = db.query(User).filter_by(username=username).first()
    if not db_user:
        raise HTTPException(detail="用户名不存在", status_code=400)
    # 使用crypt.verify校验密码是否正确
    if not crypt.verify(password, db_user.password):
        raise HTTPException(detail="用户名或密码错误", status_code=400)
    return db_user
```

补充：校验明文密码和数据库中密文密码匹配的原理

- hash的原理：使用相同的加密算法，相同的明文加密后得到相同的密文。
- 把用户的明文密码通过相同的加密算法得到的密文，和数据库中的密文相比较，相同则说明密码正确。

9-5 记录用户登录状态的方式

需求场景

对于电商购物网站，网站知道当前浏览网站的用户是谁，该用户购物车有哪些商品，该用户买了哪些商品等等。

这些需求的一个核心点就是，网站的服务端需要知道当前用户是谁。

遗憾的是，HTTP协议是无状态的，即服务端不会记录客户端的每一次请求，即每一个请求对服务端来说都是 ″陌生人″。

解决方式

计算机本身无法判断坐在显示器前的使用者的身份。进一步说，也无法确认网络的那头究竟有谁。可见，为了弄清究竟是谁在访问服务 器，就得让对方的客户端自报家门。

解决方式：用户登录认证。

核心原理：用户登录之后就给用户一个身份标识，客户端再次访问服务端时带上这个身份标识，那服务端就知道该用户的身份。

常用的具体实现方式：

- 在请求头中携带唯一标识
- 使用cookie
- 使用jwt

9-6 使用请求头实现登录认证

上节课，我们了解了一些登录认证的方式，也就是让客户端自报家门的方式，从这节课开始，我们就来具体实践。

在请求头中实现登录认证的方式，其实很简单，具体实现逻辑如下：

- 当用户访问后端服务时，后端判断请求头中是否有指定的请求头键值对，有且正确则该用户是登录过的。
- 如果请求头中没有指定的键值对，则是无效的请求。

示例：请求头中有合法的键值对才可以获取访问的图书信息。

- 当请求中没有 x-token时，或者它的值不是指定的合法token时，都校验会校验失败，得不到数据。
- 用户在登录之后才能获得一个x-token，然后请求其他接口时在请求头中携带者x-token才能获取数据。
- 即只有携带这个正确的x-token才是经过登录认证的用户，才允许获取数据。

```
X_TOKEN = "SDNQOEFJQIEVNFWESCVMWE"


@app.post("/login", response_model=UserOut)
def login(response: Response,  username: str = Form(), password: str = Form(),
db: Session = Depends(get_db)):
    db_user: User = db.query(User).filter_by(username=username).first()
    if not db_user:
        raise HTTPException(detail="用户名不存在", status_code=400)
    if not crypt.verify(password, db_user.password):
        raise HTTPException(detail="用户名或密码错误", status_code=400)

    # 第四章学的如何设置响应头哈
    response.headers["x-token"] = X_TOKEN
    return db_user


@app.get("/books")
def books(x_token: typing.Optional[str] = Header()):
    if x_token and x_token == X_TOKEN:
        return [{"id": i, "name": f"book{i}"} for i in range(1, 11)]

    raise HTTPException(detail="Invalid x_token", status_code=404)
```

9-7 使用Cookie实现登录认证

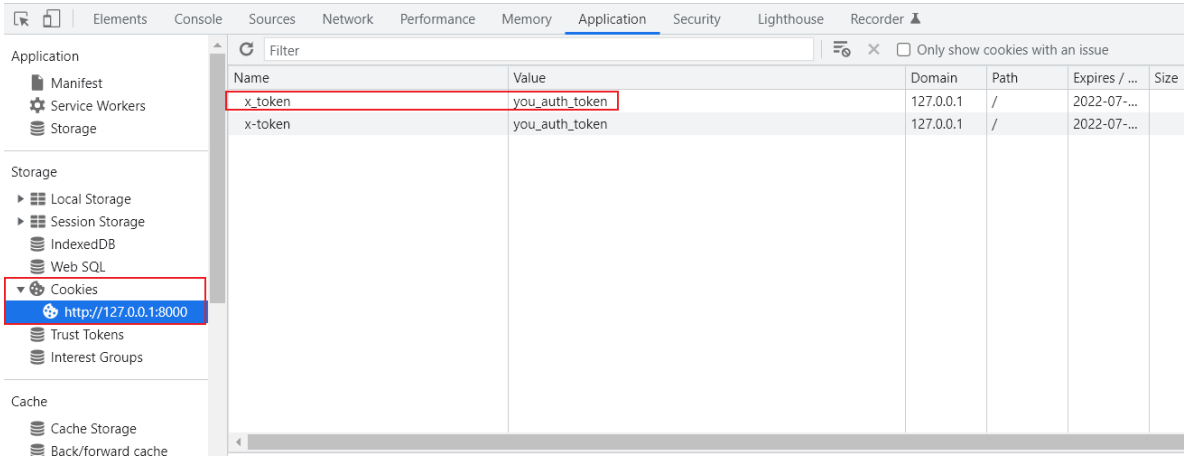
Cookie

- 储存在用户本地终端上的数据
- Cookie是一段不超过4KB的小型文本数据，由一个名称（Name）、一个值（Value）和其它几个用于控制Cookie有效期、安全性、使用范围的可选属性组成
- 只要是有效的Cookie，浏览器下次访问服务器时，就会在请求中携带cookie中的数据。

示例1：获取和设置cookie

- 在谷歌浏览器中打开F12查看NetWork和Applications中的Cookies
- set_cookie时可以value可以是一个单纯的字符串，也可以是一个被json序列化的的Python数据结构，比如字典。

```
from fastapi import FastAPI, Response, Cookie


app = FastAPI()


@app.get("/set")
def set_cookie(response: Response):
    # 设置cookie, 需要 key和value, 还可以设置过期时间（单位:秒）
    response.set_cookie("x_token", "you_auth_token", 60 * 60)
    return "success"


@app.get("/get")
# 获取cookie, Cookie()的用法和Path()\Header()等类似
def get_cookie(x_token: str = Cookie()):
    return {
        "x-token": x_token
    }
```



示例2：使用cookie做登录认证

- 登录时设置cookie，使用用户名当cookie的值
- 访问图书资源时，携带cookie，通过cookie中的用户名，可以知道当前用户的身份信息。

```
from fastapi import  Cookie, Response

...


@app.post("/login", response_model=UserOut)
def login(response: Response,  username: str = Form(), password: str = Form(),
db: Session = Depends(get_db)):
    db_user: User = db.query(User).filter_by(username=username).first()
    if not db_user:
        raise HTTPException(detail="用户名不存在", status_code=400)
    if not crypt.verify(password, db_user.password):
        raise HTTPException(detail="用户名或密码错误", status_code=400)

    # 在响应中设置cookie,
    response.set_cookie("x_token", db_user.username, 60 * 60)
    return db_user


@app.get("/books")
def books(x_token: typing.Optional[str] = Cookie(),  db: Session =
Depends(get_db)):
    if not x_token:
        raise HTTPException(detail="Invalid x_token", status_code=404)

    db_user: User = db.query(User).filter_by(username=x_token).first()
    if not db_user:
        raise HTTPException(detail="Invalid x_token", status_code=404)

    return [{"id": i, "name": f"book{i}"} for i in range(1, 11)]
```

Cookie的优点和缺点：

- 优点：简单方便

- 缺点：不安全，cookie信息可能被篡改。

9-8 使用jwt登录认证-理论篇

使用Cookie的方式做登录认证，简单方便，但是存在安全隐患，一般还不推荐使用。

Cookie之所以不安全的根本原因是：用户信息直接存放在浏览器，在网络传输中既有可能被非法篡改。

所以，后来出现了把用户信息存放在服务器端的技术Session，不过session也就基于cookie实现的。

- Session的处理逻辑如下：
- 用户登录成功后，服务端会将用户的身份信息（用户ID, 邮箱等唯一信息）加密成一个随机字符串sessionid，将sessionid和用户信息做一个映射关系，存在服务器的数据库中。
- 然后，把sessionid返回给浏览器，保存在cookie中，所以session技术是基于cookie的。
- 最后，当浏览再次请求服务器时，携带这个sessionid。服务端会校验这个session是否在数据库中有关联用户，以及是否过期等校验，校验通过后才是合法用户，才可以获取请求数据。

但是，session技术因为需要把信息存放在服务器，这样会造成维护成本高，且不容易做分布式服务。并且，一旦sessionid泄露，也是不安全的。

因为sessionid的限制，后来又出现了目前比较流行的解决方案，那就是 JWT（json web token）

JWT的解决思路：

- **首先，jwt的方案中，用户的身份标识继续使用cookie的技术，并把数据存在浏览器本地，服务端不存数据。**
- **服务器不存数据，服务器但只负责签发jwt技术产生的token（这个token一般称之为jwttoken），和校验jwttoken的合法有效性。**

- 具体表现为：用户登录成功后，服务端通过jwt的方式生成jwttoken，把这个存放浏览器的cookie中；下次请求时携带这个jwttoken，服务端负责校验jwttoken的合法性，并从中获取用户的身份信息。

JWT的生成token格式如下，由 `.` 连接的三段字符串组成。


```
eyJhbGciOiJqUadI1ASiIsInR5cCI6IkpXC8.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG
4gRG9lIiwiaWF0IjoxNTE2MjM5MDIyfQ.SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c
```


JWT token是如何签发流程：

- 第一段HEADER部分，固定包含算法和token类型，对此做JSON序列化并进行base64url加密。

  ```
  {"alg": "HS256",  "typ": "JWT"}
  ```

- 第二段PAYLOAD部分，包含一些数据，对此做JSON序列化并进行base64url加密。

  ```
  {"sub": "1234567890", "user_id": 20202, "name": "liuxu",  "email":
  "liuxu@as.com"  ...}
  ```

- 第三段SIGNATURE部分，把前两段通过`.`拼接起来，然后对其进行`HS256`加密，再然后对`hs256`密文进行base64url加密，最终得到token的第三段。对`hs256`加密时一般会做加盐处理。
- 最后将三段字符串通过 `.`拼接起来就生成了jwt的token。

JWT token的校验和提取用户信息流程：

- 与签发流程相反

9-9 使用jwt登录认证-实战篇

- 使用第三方模块实现 jwttoken的生成和校验，使用前先下载安装：`pip3 install pyjwt`

```
import jwt


secret_key = "加盐的秘钥"
data = {"user_id": 20202, "name": "liuxu",  "email": "liuxu@as.com"}


# 生成jwttoken
jwt_token = jwt.encode(payload=data, key=secret_key)
print(jwt_token)
```

```
# 解析jwttoken得到用户信息
raw_data = jwt.decode(jwt_token, key=secret_key, algorithms="HS256")
print(raw_data)
```

代码解析：

- 使用 `jwt.encode()` 签发token

```
def encode(
    self,
    payload: Dict[str, Any],        # 是需要加密的用户基本信息，比如id等，不要把用户密码放在里面
    key: str,                       # 加密时使用的秘钥，解密时也需要它
    algorithm: Optional[str] = "HS256",     # 加密的算法，默认是HS256，解密时需要使用相同的算法
    headers: Optional[Dict] = None,         # jwttoken中的第一部分，不指定时默认是{"typ": "JWT", "alg": algorithm}
    json_encoder: Optional[Type[json.JSONEncoder]] = None,
) -> str:
    pass
```

- 使用 `jwt.decode` 解析token

```
def decode(
    self,
    jwt: str,                               # jwt token
    key: str = "",                           # 加密时使用的秘钥
    algorithms: Optional[List[str]] = None,    # 加密时使用的算法，必须设置
    options: Optional[Dict] = None,
    **kwargs,
) -> Dict[str, Any]:
    pass
```

- 给token设置过期时间，在payload中增加 `exp` 字段，它的值是一个datetime对象或者时间戳

```
import jwt
import datetime

secret_key = "加盐的秘钥"
data = {"user_id": 20202, "name": "liuxu", "email": "liuxu@as.com",
        "exp": datetime.datetime.now() + datetime.timedelta(days=1)}


jwt_token = jwt.encode(payload=data, key=secret_key)
raw_data = jwt.decode(jwt_token, key=secret_key, algorithms="HS256")
print(raw_data)
```

- 校验token失败时的处理
- jwt库中提供了非常多了校验失败的错误，我们可以直接使用。或者直接使用Exception也可以。

```
import jwt
import datetime
```

```
secret_key = "加盐的秘钥"
data = {"user_id": 20202, "name": "liuxu", "email": "liuxu@as.com",
        "exp": datetime.datetime.now() + datetime.timedelta(days=-1)}


jwt_token = jwt.encode(payload=data, key=secret_key)
try:
    raw_data = jwt.decode(jwt_token, key=secret_key, algorithms="HS256")
    print(raw_data)
except jwt.ExpiredSignatureError as e:
    print(e)
```

9-10 fastapi集成jwt

需求：使用jwt做登录校验，在登录成功后给用户签发jwttoken，在图书接口中校验jwttoken，校验失败则报错，校验成功返回数据

示例1：使用pyjwt

```
# main.py
import jwt
import datetime


JWT_SECRET_KEY = "ASDN*^n23^$:_};pYz7I"


@app.post("/login", response_model=UserOut)
def login(response: Response,  username: str = Form(), password: str = Form(),
db: Session = Depends(get_db)):
    db_user: User = db.query(User).filter_by(username=username).first()
    if not db_user:
        raise HTTPException(detail="用户名不存在", status_code=400)
    if not crypt.verify(password, db_user.password):
        raise HTTPException(detail="用户名或密码错误", status_code=400)

    # 签发jwttoken，并保存在响应头上
    exp = datetime.datetime.now() + datetime.timedelta(days=1)
    jwt_token = jwt.encode(payload={"id": db_user.id, "name": db_user.username,
"exp": exp}, key=JWT_SECRET_KEY)
    response.headers["x-token"] = jwt_token
    return db_user


@app.get("/books")
def books(x_token: typing.Optional[str] = Header()):
    # 请求头取取 x-token，解析token并获取用户信息
```

```
    try:
        data = jwt.decode(x_token, key=JWT_SECRET_KEY, algorithms="HS256")
        return {
            "msg": f"welcome: {data['name']}",
            "books": [{"id": i, "name": f"book{i}"} for i in range(1, 11)]
        }
    except Exception as e:
        raise HTTPException(detail=e, status_code=404)
```

示例2：使用第三方包 python-jose来签发和校验jwttoken，使用前先安装：`pip3 install "python-jose[cryptography]"`

```
import datetime

from jose import jwt, JWTError


SECRET_KEY = "09d25e094faa6ca2556c818166b7a9563b93f7099f6f0f4caa6cf63b88e8d3e7"
JWT_EXP = datetime.datetime.now() + datetime.timedelta(days=1)

data = {"name": "liuxu", "exp": JWT_EXP}
jwt_token = jwt.encode(data, SECRET_KEY, algorithm="HS256")
print(jwt_token)

try:
    data = jwt.decode(jwt_token, SECRET_KEY, "HS256")
    print(data)
except JWTError as e:
    raise e
```

9-11 fastapi的登录认证工具

需求场景

现在我们已经可以使用jwt做登录认证了，但是你会发现在开发的时候，这个流程比较麻烦。

比如，你在api文档页面操作：

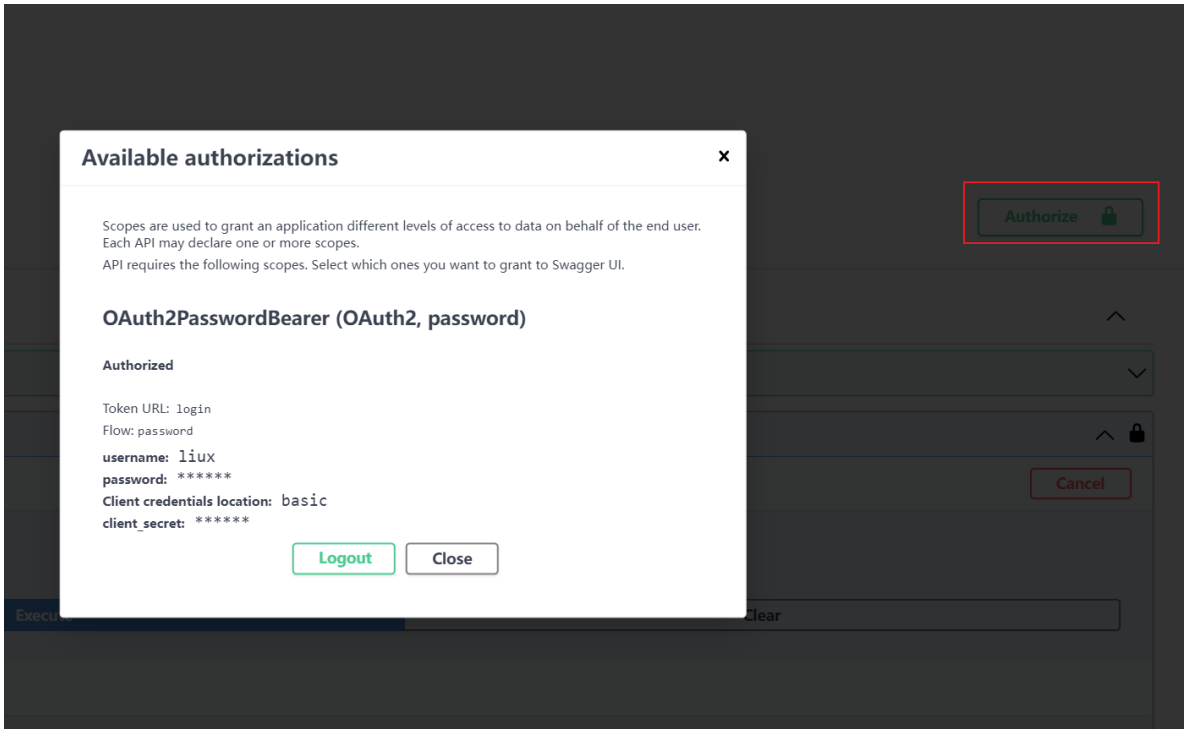
- 1 先调用登录接口，登录后在响应头中把x-token复制出来
- 2 然后调用图书接口时，把复制出来的jwttoken贴在请求上
- 3 一个图书接口操作一遍还行，但是如果有很多接口都依赖登录，就会很麻烦。那有比较优雅的解决方式吗？

FastAPI的解决方式

- 使用OAuth2PasswordBearer，实现自动登录认证

```
from datetime import datetime, timedelta
from fastapi import FastAPI, Form, Depends, HTTPException
from fastapi.security import OAuth2PasswordBearer
from jose import jwt, JWTError


app = FastAPI(title="XXX项目文档")

# 第一步：实例化对象oauth2_scheme， tokenUrl="login"表示依赖的登录接口是 "/login"
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="login")


SECRET_KEY = "09d25e094faa6ca2556c818166b7a9563b93f7099f6f0f4caa6cf63b88e8d3e7"


@app.post("/login")
def login(username: str = Form(), password: str = Form()):
    if not username or not password:
        raise HTTPException(detail="wrong username or password",
status_code=404)

    exp = datetime.utcnow() + timedelta(days=1)
    access_token = jwt.encode({"sub": username, "exp": exp}, SECRET_KEY,
"HS256")

    # 第二步：登录成功后返回 access_token和token_type两个字段
    # access_token是jwttoken，token_type="bearer"
    return {"access_token": access_token, "token_type": "bearer"}


@app.get("/books")
def get_books_list(token: str = Depends(oauth2_scheme)):
    # 第三步：需要登录后才能使用的接口，使用依赖注入Depends(oauth2_scheme)
    # oauth2_scheme会帮我们解析出jwttoken，并赋值给形参token
    try:
        data = jwt.decode(token, key=SECRET_KEY, algorithms="HS256")
        return {
            "msg": f"hello: {data['sub']}",
            "books": [{"id": i, "title": f"book{i}"} for i in range(5)]
        }
    except JWTError as e:
        raise e
```

下图是openadpi文档的登录认证交互窗口，有了它我们就不必再手动调用登录接口，复制token了



补充：

- OpenAPI文档不支持携带特殊的请求头字段，但是自定义的请求头是OK的

9-12 fastapi登录认证工具的内部原理

带着大家看看源码

9-13 登录相关工具封装

- 密码加密和校验封装成一个工具
- jwt签发和解析封装成一个工具

```
# tools.py

import datetime

from passlib.context import CryptContext
from jose import jwt, JWTError
```

```python
class Hashing:
    def __init__(self, schemes: str = "bcrypt"):
        self.crypt = CryptContext(schemes=[schemes], deprecated="auto")

    def hash(self, raw_pwd: str) -> str:
        return self.crypt.hash(raw_pwd)

    def verify(self, raw_pwd: str, hashed_pwd: str) -> bool:
        return self.crypt.verify(raw_pwd, hashed_pwd)


class Jwt:
    JWT_KEY = "ASDN*^n23^$:_};pYz7I"
    ALGORITHMS = "HS256"

    def set_token(self, data: dict) -> str:
        if "exp" not in data:
            data["exp"] = datetime.datetime.now() + datetime.timedelta(days=1)
        return jwt.encode(data, key=self.JWT_KEY)

    def get_token(self, jwt_token: str):
        try:
            return jwt.decode(jwt_token, self.JWT_KEY, self.ALGORITHMS)
        except JWTError as e:
            raise e


hash_obj = Hashing()
jwt_obj = Jwt()
```

- 使用时，直接导入

```python
from tools import hash_obj, jwt_obj

app = FastAPI(title="登录认证相关")


@app.post("/register", response_model=UserOut)
def register(user: UserForm = Depends(), db: Session = Depends(get_db)):
    ...
    new_user = User(
        username=user.username,
        password=hash_obj.hash(user.password),      # hash加密
        email=user.email
    )
    ...


@app.post("/login", response_model=UserOut)
def login(response: Response,  username: str = Form(), password: str = Form(),
db: Session = Depends(get_db)):
    db_user: User = db.query(User).filter_by(username=username).first()
    if not db_user:
```

```python
        raise HTTPException(detail="用户名不存在", status_code=400)

    # hash校验
    if not hash_obj.verify(password, db_user.password):
        raise HTTPException(detail="用户名或密码错误", status_code=400)
    # jwt签发
    jwt_token = jwt_obj.set_token({"id": db_user.id, "name": db_user.username})
    response.headers["authorization"] = jwt_token
    return db_user


@app.get("/books")
def books(authorization: typing.Optional[str] = Header()):
    # jwt解析
    data = jwt_obj.get_token(authorization)
    return {
        "msg": f"welcome: {data['name']}",
        "books": [{"id": i, "name": f"book{i}"} for i in range(1, 11)]
    }
```